

TDOM: TeX のための Document Object Model と 無改造 LuaTeX における検証つき増分組版

TDOM: A Document Object Model for TeX — Verified Incremental Typesetting on
Unmodified LuaTeX

川口 晃世 幸川 卓実

株式会社 Fermion

`contact@fermion.company`

2026 年 8 月 18 日

概要

L^AT_EX による執筆では、わずかな修正であっても文書全体の再コンパイルを経なければ結果を確認できず、その待ち時間は文書の成長とともに増大して執筆の反復を妨げる。本稿は、かつて Web ブラウザが静的ページの再読込を DOM(Document Object Model) に基づく増分再描画へ置き換えたのと同じ移行を、無改造の LuaTeX の上で実現する組版ランタイム TDOM(TeX Document Object Model) を提案する。T_EX のマクロ言語は実行中に字句規則を変更しうるため、Web におけるような構文木を静的に定めることはできない。そこで TDOM は文書をブロックの列に分割し、各ブロック境界まで処理を終えた TeX プロセスの全状態を `fork(2)` の copy-on-write スナップショットとして保持することで、構文木の代わりに実行状態の列を文書の内部表現とする。編集は変更ブロックとその影響範囲の再組版として処理され、改ページ・フロート・脚注・相互参照・目次を含む大域的なレイアウトも、全文の再コンパイルを待たずに増分的に収束する。表示が通常の LuaLaTeX 出力と一致することは、PDF コンテンツストリームに基づく 0.1 bp 精度のレイアウト照合、および無作為な編集列の後で増分状態が新規構築状態と一致することの検査を含む 4 系統の手続きで機械的に確認され、一致を保証できない構文は行・ブロック・文書の各粒度で実出力による表示へ退避する。評価では、4 ページから 30 ページの文書に対して定常の打鍵応答が p50 で 12~25.2 ms、p95 で 100 ms を下回り、文書規模にほとんど依存しないことを確認した。

Abstract. Editing a L^AT_EX document still requires recompiling the entire file before any change can be inspected, and the delay grows with the document. This paper presents TDOM (TeX Document Object Model), a typesetting runtime that carries over to unmodified LuaTeX the transition the Web once made from page reloads to incremental re-rendering over the DOM. Because the T_EX macro language can alter its own lexical rules during execution, no syntax tree of the source can be defined statically; TDOM therefore represents the document as a sequence of blocks whose boundary states are complete TeX processes, preserved as `fork(2)` copy-on-write snapshots. An edit is processed as retypesetting of the changed block and its bounded range of influence, while page breaking, floats, footnotes, cross-references and the table of contents converge incrementally rather than awaiting a full recompilation. Agreement between the display and genuine LuaLaTeX output is checked mechanically by four independent procedures, including layout comparison at 0.1 bp precision against PDF content streams and an equivalence test between incrementally maintained and freshly built engine states under randomized edits; constructs whose agreement cannot be established fall back to display taken from the real output, at line, block or document granularity. In our evaluation, steady-state keystroke response remains at 12–25.2 ms (p50) and below 100 ms (p95) on documents of 4 to 30 pages, largely independent of document size.

1 はじめに

L^AT_EX を用いた執筆では、原稿に手を入れるたびに文書全体をコンパイルし直し、生成された PDF を開いて該当箇所を確かめるという手順が繰り返される。コンパイルの所要時間は文書の規模とともに延び、パッケージを多用する 100 ページ規模の書籍では 1 回あたり 10～30 秒に達することが報告されている [1]。この待ち時間は一回ごとに見れば小さいが、修正と確認のあいだに毎回挟まることで執筆の反復そのものを重くし、書き手は結果の確認を後回しにするか、確認のたびに思考を中断するかを選択を迫られる。組版品質の点で L^AT_EX に代わるものがない分野ほど、この編集と表示の断絶は避けがたい負担であり続けてきた。

同種の断絶が解消された先例として、Web の変遷が参考になる。1990 年代の Web ページは操作のたびに全体をサーバから読み直すものであったが、DOM(Document Object Model)[12] の標準化を境に、文書はブラウザ内に保持されるオブジェクトの木となり、スクリプトによる木の変更に對してレイアウトエンジンが影響範囲だけを再計算して再描画する構成へと移行した。ページの再読込が木の変更に置き換えられたことは単なる高速化にとどまらず、のちの対話的な Web アプリケーションが成立する基盤となった。

T_EX についても、遅さの原因が組版アルゴリズムそのものにはないことは実測から知られている。Knuth–Plass の行分割 [3] は 1 段落あたり 1 ms 程度で完了する [1]。時間を要しているのはむしろ、入力を読み、全体を処理し、出力を書き、内部状態を残さず破棄するという使い捨てのプロセスモデルの方であって、このモデルを保ったままでは、キャッシュや並列化を導入しても所要時間は文書規模への比例を脱しない。文書を実行中のランタイムの内部に保持し、編集を保持された状態への変更として扱うことができれば、組版品質に触れることなく、編集応答だけを対話的な水準へ引き下げられるはずである。

本稿で提案する TDOM(TeX Document Object Model) は、この構成を LuaTeX の上で実現するものである。ただし、Web の DOM を T_EX へそのまま持ち込むことはできない。T_EX のマクロ言語は実行中に字句規則 (catcode) 自体を書き換えられるため、ソースの静的な解析によって

構文木を得ることが一般にはできないからである [4]。TDOM はこの制約を、木のノードにパース結果ではなく実行状態を対応させることで回避する。ブロック k に対応するノードの実体は、ブロック $1..k$ を組版し終えた時点の TeX プロセスそのものであり、POSIX の `fork(2)` が提供する copy-on-write 機構によって、マクロもカウンタもフォントも catcode も含む全状態が一つのスナップショットとして保持される。個々の状態を列挙して保存・復元する必要はない。

編集された箇所だけを再コンパイルするという方向は、近年独立に追求されている。Lode[1] は行分割が段落の外部に依存しないことに着目し、編集された 1 段落だけを常駐 LuaTeX で再コンパイルすれば約 1 ms で完了することを示した。この結果は本研究の前提を実測の面から裏づけるものであるが、同システムでは段落が文書の文脈から切り離してコンパイルされるため、カウンタや相互参照、周辺コードの設定する `\parshape` 等に依存する内容は高速経路の対象にならず、改ページ・フロート・脚注・ページ番号の更新は周期的に走る全文再コンパイルに委ねられる。さらに、高速表示が実際の L^AT_EX 出力と一致していることを確かめる機構は備えていない。本研究が扱うのはこの三点、すなわち文脈の保存、大域効果の増分的な収束、出力の機械的な検証である。

本稿の貢献は次の四点に要約される。第一に、素の LuaTeX に 71 行の C シムをロードするだけで、fork checkpoint により文書の実行状態の列を維持する構成を示す (§4)。エンジンの改造を必要とした TeXpresso[2] と異なり、既存の CTAN 資産にも将来の TeX Live 更新にもそのまま追従できる。第二に、各ブロックの出口状態ベクトルと組版結果ハッシュに基づく増分再組版を示す (§4)。編集時の同期処理は有界個数のブロック検証で打ち切られ、残りは非同期に収束するため、相互参照・目次・改ページ・フロート・脚注が全文再コンパイルを待たずに更新される。第三に、表示の正しさそのものを検査対象とする構成を示す (§5)。表示は 3 値の忠実度判定に従い、判定できない行は実出力へ退避したうえで、表示と LuaLaTeX 実出力の一致が 4 系統の手続きで機械的に検査される。第四に、実測により、定常打鍵応答が文書規模にほとんど依存せず、すべての規模と編集位置で p95 が 100 ms を下回ることを示す (§6)。

2 背景

2.1 バッチ型のプロセスモデル

`lualatex main.tex` の 1 回の実行は、プロセス起動とフォーマット読込に約 0.3 秒、プリアンブル処理に 0.1~5 秒を要したのち、本文を先頭から末尾まで逐次組版し、PDF を書き出して内部状態を破棄する。これに多重パスの問題が重なる。相互参照や目次のページ番号は組版後にしか確定しないから、L^AT_EX は 1 回目の実行で参照情報を補助ファイル (aux) に書き出し、2 回目以降の実行でそれを読み戻して正しい番号を印字する。したがって正しい出力を得るには最低 2 回、目次がページ数を動かす場合には 3 回の全文コンパイルが必要になる。編集のたびにこの手順を繰り返すという点で、今日の L^AT_EX 執筆は Web におけるページ再読込と同じ形のループを保ったままである。

表1 リアルタイム L^AT_EX への先行アプローチと TDOM

	エンジン改造	増分の単位	文脈保存	大域のライブ収束	出力の機械検証
latexmk 等の自動再実行	不要	文書全体	—	(全文)	なし
TeXpresso[2]	必要	編集位置以降	あり	再実行で収束	なし
SwiftLaTeX/BusyTeX	不要	文書全体	—	(全文)	なし
Typst[6]	(新エンジン)	式単位・ $O(n)$	あり	毎回全体再収束	なし
Lode[1]	不要	段落・ $O(1)$	なし	全文コンパイル待ち	なし
TDOM(本稿)	不要	ブロック・有界	あり	ライブ	あり (4 系統)

表2 Web の DOM と TDOM の対応

	Web ブラウザ	TDOM
文書の内部表現	DOM ツリー (パース結果)	ブロック列 + fork 済み TeX プロセス (実行状態)
変更の単位	ノード・属性	ブロック (<code>\par</code> 境界・環境は不可分)
影響範囲の追跡	スタイル無効化・dirty bit	出口状態ベクトル + 組版結果ハッシュ + 依存表
再レイアウト	増分 reflow	有界の同期再組版 + 非同期収束
再描画	差分ペイント	display list のページ単位パッチ
最終的な出力	(レンダラ自身)	通常の LuaLaTeX 出力 (canonical 層)

2.2 関連研究

主な先行アプローチを表 1 に整理する。TeXpresso[2] は XeTeX エンジン改造し、プロセスの `fork(2)` をチェックポイントとして編集位置以降を再実行するもので、実行状態をプロセスとして保存する点は本研究と共通するが、エンジン本体への改造を必要とする点で異なる。SwiftLaTeX[8] と BusyTeX[9] は T_EX を WebAssembly へコンパイルしてブラウザ内で実行するものであり、起動や配布の形態は変わるものの、コンパイルそのものは毎回全文である。Typst[6, 7] は増分計算を言語設計に組み込んだ新しい組版系であるが、L^AT_EX との互換性を持たず、また増分再コンパイルの時間は文書規模に対して線形に増加することが報告されている [1]。Lode[1] は前節で述べたとおり無改造 LuaTeX における段落単位の再コンパイルを示したが、文脈の保存、大域効果の増分更新、出力の検証は扱っていない。

3 設計

TDOM の名称は Web の DOM に由来し、その対応は表 2 に示すとおり構成の各層に及ぶ。DOM が HTML のパース結果を保持するのに対して、TDOM が保持するのは実行状態である。この違いは前節で述べた T_EX の言語的性質からの帰結であって、それを除けば編集ループの構造は同型である。すなわち、木の一部が変更されると、変更されたノードとその影響範囲だけが再計算され、表示には差分のみが送られる。

設計にあたっては、次の三点を方針とした。

第一は文脈の保存である。ブロックを文書から切り離して単体でコンパイルすることはせず、ブロック k の組版は、ブロック $1..k-1$ を処理し終えたプロセスから fork した子が実際の main

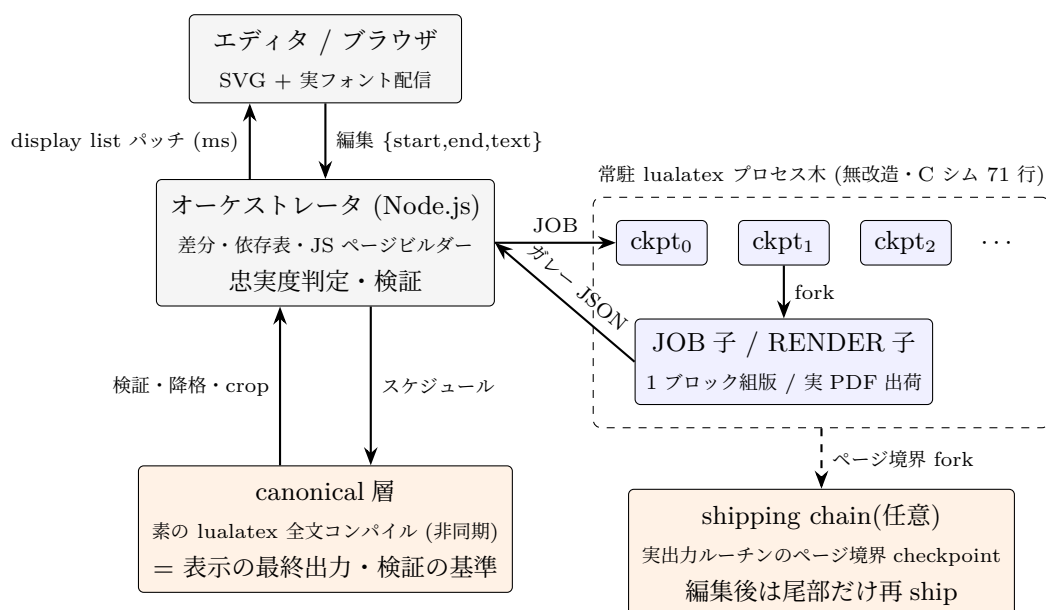


図1 TDOM の全体構成。常駐プロセス木が保持された実行状態の実体であり、canonical 層とその増分化である shipping chain が最終的な出力を供給する。

vertical list 上で行う。この構成では、カウンタ、マクロ定義、フォント状態、`\prevdepth`、段落間空白の max 合成といった文脈依存の要素が構成上正しくなり、収穫されるノードストリームは連続実行の結果とバイト単位で一致する。

第二は表示の検証である。高速な表示は、通常の LuaLaTeX 出力と一致すると判定できた範囲に限って用い、判定できない行は実出力から生成した画像で置き換える。さらに事後の照合で乖離が確認された領域は自動的に降格させる。表示が一時的に古いことは許容するが、実出力と異なる表示は許容しないという規律で全体を構成した。

第三は段階的な退避である。CTAN に蓄積された膨大な資産のすべてを高速経路で扱うことは現実的でないから、扱えない構文は、行 (数式行の画像化)、ブロック (隔離コンパイル)、文書 (canonical 表示のみ) の 3 粒度で退避させる。どの粒度に退避してもソースの編集は継続できる。

4 アーキテクチャ

4.1 fork checkpoint

常駐ルートは通常の lualatex として起動され、その中の `\directlua` から 71 行の C シムを `package.loadlib` でロードする。このシムが公開するのは `fork`、`waitpid`、`_exit` 等の数個のシステムコールにすぎず、エンジンの実行ファイルにも LaTeX カーネルにも変更を加えない。ただし共有ライブラリの読込は制限モードで禁じられているため、常駐ルートの起動にのみ `--shell-escape` を要する。ブロック境界 k の checkpoint はブロック 1.. k を処理し終えたプロセスであり、編集の際には最寄りの checkpoint から子を fork し、編集されたブロックを注入して組版させ、完了した子が次の checkpoint となる。プロセス数はグリッド間隔と編集位置のピン留めによって上限管理され (既定は最大 64、後述の検証実行時は 8)、疎なグリッドから再開する場合の追加費用はクリーン

ブロック 1 個あたり約 3 ms の再組版にとどまる。

4.2 ノードストリームの収穫

各ブロックの組版結果は、PDF を経由することなく、 $\text{\TeX}$ 内部の main vertical list、すなわち `tex.lists.page_head` から直接取り出す。このあいだページビルダーは休止させておく。すなわち `\vsize` を `\maxdimen` に設定し、脚注は `\holdinginserts=1` によって本文の流れに保持したまま、行ボックス、伸縮量と次数を含むグルー、ペナルティ、脚注インサート、フロートアンカーを JSON として収穫する。グルーの実効値は Lua $\text{\TeX}$ の `node.effective_glue` で解決するから、得られるグリフ座標は出荷される PDF のそれと一致する。この抽出が改行の後かつ改ページの前で行われる点が本設計の要であり、改ページの入力となるグルーとペナルティが保存されたまま手に入るからこそ、次節のページビルダーが $\text{\TeX}$ と同じ判断基準で改ページを再実行できる。

4.3 ページビルダー

収穫したストリームに対しては、JavaScript 側のページビルダーが $\text{\TeX}$ のページ分割と L $\text{\TeX}$ の出力ルーチンに相当する処理を編集のたびに実行する。badness 計算は `tex.web` §108 に対応する移植であり、フロート配置、脚注の組み付け、`\topskip`、柱とノンブル、`\cleardoublepage` の偶奇処理と空白ページの挿入までを扱う。`\topfraction` などのパラメータはすべて実行中の $\text{\TeX}$ から実測値として取得しており、この層が独自の寸法を持ち込むことはない。ページ境界のスナップショットを保持しているため、再ページ付けの費用は編集近傍のページ数に比例する範囲に収まる。Lode[1] が改ページの更新を周期的な全文コンパイルに委ねたのに対し、TDOM では改ページも編集と同じ応答の中で更新される点が異なる。

4.4 影響範囲の計算

各ノードは組版結果のハッシュと出口状態ベクトル (全カウンタ、`\prevdepth`、`\if@nobreak`、末尾の `\lastskip`) を持つ。編集時の同期処理では、編集されたブロックに続くクリーンブロックを、出力と状態が完全に再現されることを確認するまで再組版する。ただしこの検証は有界であり、レイアウトが結合しているブロックについて 8 個、局所状態の波及について 4 個で打ち切る。打ち切り時点の判定は 3 値である。収束を確認すれば `clean`、カウンタ等のみが動いていれば `counters`、マクロ定義の変更などによって下流を信用できなければ `leak` と判定し、後二者の残余は編集応答に含めず、編集が 300 ms 途絶えてから走る非同期の収束処理 (`counters` に対する `settle`、`leak` に対する `rebuild`) に委ねる。この処理は次の編集が来れば直ちに中断し、進捗位置から再開する。相互参照については、aux ファイルの代わりに `\r@...` マクロを実行時に定義するラベル表と参照インデックスで追跡し、後方参照の再組版と目次の不動点処理 (最大 3 パス) も同じ枠組みの中で収束させる。

4.5 忠実度判定と exact chunk

ブラウザでのグリフ描画には、 $\text{\TeX}$ が実際に使用したフォントファイルをそのまま配信し、ブラウザ側の整形 (カーニングとリガチャ) を無効化したうえで、行内の位置をフォントの字送りから再現する方式を用いる。それでも一致を保証できない行が残る。ブラウザの読めない Type1 数式フォントや cmap 外のサイズバリエーションを含む数式行、TikZ などの PDF リテラルを含む行がそれであり、これらは行単位で実出力の画像、すなわち checkpoint 子が `\shipout` した単ページ PDF を SVG 化したものに置き換える。出力ルーチン自体に介入する環境 (`multicols`、`longtable`、分割可能な `tcolorbox` など) と、ページの残り高さを読んで自らの配置を決める環境 (`wrapfigure` や横倒しのフロートなど) は、いずれもブロック単位の隔離コンパイルへ退避させる。その際にはページ上の開始位置を 0.25 bp 単位で再現したうえで、配置と分割の判断を実際の実出力ルーチンに行わせる。文書全体に影響する構文 (`shipout` フック、`twocolumn` など) を含む場合は文書単位で canonical 表示へ退避する。いずれの退避においても、ソースの編集が中断されることはない。

4.6 canonical 層と shipping chain

表示の最終的な出力は常に、`lualatex` を aux 不動点まで実行した通常のコmpイル結果である (canonical 層)。このコンパイルは編集応答に含めないだけでなく、編集の合間ごとに走らせることもしない。最初の一回だけは短い遅延 (既定 2.5 秒) で基準を作り、以後は連続した静止 (既定 30 秒) と、直前のコンパイル費用に比例するクールダウンの双方を満たしたとき、あるいは PDF 出力が明示的に要求されたときに限って実行する。ライブの表示は provisional 層が担っているのだから、canonical 層は書き手が一区切りつけた時点での確認と、次節で述べる検証の基準として位置づけられる。コンパイルが完了すると、変更のないページは LuaLaTeX 自身の出力画像に置き換わり、同時に検証が走る。さらに任意機能として、実際の実出力ルーチンの `shipout` ごとにページ境界 checkpoint を取る第 2 の常駐実行 (shipping chain) を備える。編集後は、最初に変更されたブロックより前のカーソルを持つページ checkpoint から末尾側だけを再 ship すればよいから、最終出力の更新に要する時間も全文の再コンパイルではなく、編集位置以降のページ数に比例する量となる。

5 検証

表示が実出力と一致するという性質を、主張ではなく検査可能な条件として扱うために、TDOM は独立な 4 系統の検証を備える (表 3)。その中心にあるのは増分状態と新規構築状態の同一性である。すなわち、任意の編集列を処理した後のエンジンの全ブロック状態 (組版結果ハッシュと状態ベクトル) は、最終ソースを新しいエンジンで開き直した結果と一致しなければならない。シードを固定したランダム編集による検査がこの条件を編集バーストごとに確認しており、開発の過程では、参照値の記帳誤りやページ文脈に依存するブロックの固着といった増分処理の欠陥が、いずれもこの検査によって発見され修正された。なお、コンパイル不能なソース (入力途中の不完全な $\text{\TeX}$) には収束すべき真値が存在しないから、このときは最後の正常状態を凍結して番号の変動を抑え、ソースの修正とともに自動的に回復させる。凍結と回復の経路もまた同じ検査の対象である。

表3 TDOM の検証系統。いずれも通常の LuaLaTeX 出力を基準とする。

検証	内容	精度	実行
レイアウト照合 (<code>verify-layout/farm</code>)	全行のベースライン座標を、実 PDF のコンテンツストリーム直読 (ラスタイザ非経由) と照合	0.1 bp	コーパス 11 文書・CI 必須
ノードストリーム照合 (<code>compare-breaks</code>)	実 $\text{T}_{\text{E}}\text{X}$ の出力ルーチン発火ごとに自然グルー値と box 寸法をダンプし、ページビルダーと突合	0.02 bp	診断時
増分同一性検査 (<code>fuzz</code>)	ランダム編集バースト後に、増分状態と新規構築状態の全ブロックを比較	完全一致	CI(シード固定)
増分正本の照合 (<code>shipping</code> 検査)	再 ship された各ページのテキストが冷間の全文コンパイルと一致	ページ単位	CI

検証は実行時にも働く。canonical 層のコンパイルが完了するたびに、表示テキストとの照合が文字バイグラムの含有率に基づいて行われ (この規則はラテン文字と CJK を同一に扱える)、確信度の高い乖離が見られたブロックだけが自動的に降格する。ブラウザ側でフォントの読込に失敗した場合もサーバへ報告され、該当ファミリを使う行は実出力画像による表示に切り替わる。このように検証は、表示経路の外側に置かれた検査ではなく、表示の選択そのものを制御する情報として系の内部に組み込まれている。

6 評価

6.1 設定

計測環境は Apple M5、メモリ 16 GB、LuaHBTeX 1.18.0 (TeX Live 2024) である。文書には `article` クラスの英文書を 3 規模用意した (S: 4 ページ・49 ブロック、M: 20 ページ・241 ブロック、L: 30 ページ・361 ブロック。各節は 4 つの段落、番号つき数式 1 つ、前方参照と後方参照を含む)。プリアンプルは `amsmath` のみの軽量の構成であり、全文コンパイル時間としては下界に近い条件にあたるから、実際の文書では以下で示す差はさらに開くことになる。エンジンは checkpoint 上限 8 で起動した。これは既定値 64 より大幅に疎な省メモリ設定であり、後述のとおり初回打鍵の再生距離を長くする、本手法にとって不利な側の設定である。canonical と精密レンダラーの非同期層は、設計上編集応答に含まれないため (§4)、本計測では無効化した。打鍵は文書の 25%、60%、92% の 3 箇所各 12 打 (1 文字挿入) を行い、同期応答 (`edit()` の往復) の分布を取った。比較の基準としては、同一文書の通常の全文コンパイル (`lualatex` 2 パス) を同一機で計測した。計測スクリプトと生データは実装とともに公開している [14]。

6.2 結果

結果を表 4 に示す。まず定常の打鍵応答は、文書規模にほとんど依存しなかった。規模が S から L へ 7 倍になっても定常打鍵は p50 で 15.4~25.2 ms、p95 でも最悪 85.6 ms にとどまり、すべての規模と編集位置で、対話的応答の目安とされる 100 ms[5] を下回った。これに対して全文コンパイ

表 4 文書規模と編集応答 (打鍵は 1 文字挿入、3 箇所 × 12 打の最悪箇所値)。「定常」は編集位置ピン確立後 (2 打目以降)、「初回」はカーソル移動直後の checkpoint グリッド再生を含む 1 打目。全文コンパイルは `lualatex` 2 パスの実測。

	S(4p)	M(20p)	L(30p)
全文コンパイル (2 パス)	0.7 s	0.8 s	0.8 s
TDOM 起動 (1 回のみ)	1.4 s	1.8 s	2.0 s
定常打鍵 p50	15.4 ms	12.5 ms	25.2 ms
定常打鍵 p95	24.6 ms	66.7 ms	85.6 ms
カーソル移動後の初回打鍵	30.8 ms	93.8 ms	122 ms
節の挿入 (番号繰り下げ)	23.5 ms	89.0 ms	102 ms
マクロ定義の変更	34.7 ms	42.0 ms	43.6 ms

ルの時間は、この軽量な文書群でも規模とともに増加する (0.7 秒から 0.8 秒)。両者の比は本計測の下界条件で約 30 倍であるが、実際の文書では全文コンパイルが 10~30 秒に達する [1] ことを考えれば、差はさらに大きい。編集のたびに要していた時間は、起動時に一度だけ要する時間 (L で 2.0 秒) に置き換えられたことになる。

応答分布の裾については要因を特定できる。カーソル移動直後の初回打鍵だけは最大 122 ms (L、中間部) を要したが、これは新しい編集位置での 1 打目が、疎な checkpoint グリッドからクリーンブロックを再生する費用を負担するためであり、2 打目以降は編集位置のピン留めによって定常値に戻る。グリッドの再生距離は上限数に反比例するから、既定の上限 64 の下ではこの距離は本計測の約 8 分の 1 になる。観測された裾は原理的な限界ではなく、メモリ使用量と初回応答とのあいだの設定可能なトレードオフである。

影響範囲の大きい編集も有界の時間で応答した。節の挿入は以降のすべての節番号と参照を動かす編集であるが、応答は 102 ms (L) であり、番号の伝播は非同期処理に委ねられて編集応答はそれを待たない。マクロ定義の変更は下流全体の再組版を要する最悪の編集にあたるが、同期応答は 43.6 ms にとどまり、再構築は背景で収束した。編集体験を決めるのは平均値だけではないから、最悪の編集で応答が有界であることには平均値と同等以上の意味がある。

最後に、以上の数値は検証に合格している実装から得られたものである。本計測と同一の設定で、数式・フロート・脚注・参照・日本語混植・rescue 環境を含む公式コーパス 11 文書のレイアウト照合は 298/298 行のベースライン照合で全数一致となり、編集ホットパス不変量のテストは 11/11 を通過した。

6.3 先行システムとの関係

Lode[1] は段落単体の再コンパイルが 0.1~2 ms、Typst の増分再コンパイルが 10、100、300 ページに対して 12.6、76、206 ms であったと報告している (いずれも同論文の計測環境による値である)。TDOM の打鍵応答はこれらと同じく文書規模に依存しない階級に属するが、計測に含まれる処理の範囲が異なる。TDOM の数値には、fork による文脈の保存、改ページの再実行、フロートと脚注の配置、参照の追跡までが含まれており、Lode の高速経路が対象外とした大域効果を処理に含めたうえで、なお応答が同じミリ秒台にとどまることを示している。この点が本稿の主要な実測

結果である。

7 応用

文書が実行状態を保持する木として管理されることの意味は、待ち時間の短縮にとどまらない。ここでは本構成の上に成り立つ応用の方向を三つ挙げる。

第一は直接編集である。TDOM の `display list` では全描画命令がソース位置への逆写像 (ブロック id とオフセット) を保持しているから、プレビュー上の任意の文字から対応するソース範囲を引くことができる。これをミリ秒の編集応答と組み合わせれば、組版結果の上で直接テキストを編集し、組版がその場で追従するというワードプロセッサ型の操作を、 $\text{\LaTeX}$ の組版品質のまま構成できる。数式についても事情は同じであり、数式行は行単位の `exact chunk` としてソース範囲に対応付けられているから、数式を選択してその $\text{\TeX}$ 記述を編集し、検証済みの組版で確認するという操作が同じ機構の上に成り立つ。WYSIWYG 編集で問題となってきた表示と最終出力の乖離は、 $\S 5$ の検証層が抑える。

第二は、大規模言語モデルに対する構造化された文脈の供給である。現状、モデルに $\text{\LaTeX}$ 文書を扱わせる際の入力には生の `.tex` 文字列か PDF であり、どちらも文書の構造を保持していない。TDOM の内部表現は、ブロック単位の構造、各ブロックの依存関係 (参照するラベルと消費する目次)、編集ごとの正確な変更集合、描画座標とソースの双方向対応を備えた中間表現であるから、変更に関係する部分木とその組版状態だけを文脈として渡すことも、モデルの提案を編集 API に適用してミリ秒で組版と検証の結果を得ることもできる。バッチコンパイルの所要時間は、このような反復をこれまで実用の外に置いてきた。

第三に、`display list` のパッチは複数クライアントへの同報が可能であるから共同編集の基盤となり、また増分正本 (shipping chain) は印刷と同一のページが編集に追従して得られるという性質から、出版工程における校正の反復にも応用できる。

8 制限

現在の実装には次の制限がある。第一に、`microtype` (文字の突出しと伸縮) には対応しておらず、使用した文書ではグリフ層の座標が微小にずれる可能性がある。検出時の退避と対応は今後の課題である。第二に、二段組、カスタム出力ルーチン、`shipout` フック系のパッケージは文書単位で `canonical` 表示へ退避する (編集は継続できる)。第三に、脚注のページまたぎ分割は未実装であり、分割が必要な場合は早い改ページで近似したうえで `canonical` 層が最終形を示す。第四に、常駐プロセス木は `fork(2)` に依存するため、ネイティブ Windows は対象外である。第五に、`checkpoint` は実プロセスであるから、メモリ使用量は規模と上限設定に比例する (本評価設定での常駐プロセス木の見かけ RSS 合計は約 417 MB であるが、これは `copy-on-write` 共有ページを各プロセスに重複計上した値であり、実効はこれより小さい)。第六に、単一 `fork` 系譜の深さには現在限界がある。120 節・721 ブロック (60 ページ) の予備実験では、系譜の劣化により組版ジョブが既定タイムアウト (12 秒) に達し、起動にも 331 秒を要した (生データは同梱の `measurements-1120.json` に収めた)。本評価の範囲 (361 ブロック) はこの限界の手前にある。これらのうち第二・第三の項目は段

階的退避の枠組みの中で対応でき、第一はグリフ層の拡張、第四は分離コンパイル型の縮退モード、第六は checkpoint の定期再生成による系譜の平坦化によって対応できると考えている。

9 結論

本稿では、文書を実行状態の列として保持する組版ランタイム TDOM を提案し、無改造の LuaTeX の上で、編集応答を文書規模からほぼ独立なミリ秒台にできることを実装と実測によって示した。編集はブロック単位の増分再組版として処理され、改ページ・フロート・脚注・相互参照・目次は全文の再コンパイルを待たずに更新される。表示と実出力の一致は 4 系統の検証によって機械的に検査され、検証できない構文は実出力による表示へ段階的に退避する。評価では、プリアンブルが軽量な下界条件においても全文コンパイル 0.8 秒に対して定常打鍵 p95 85.6 ms が得られ、全文コンパイルが 10~30 秒に達する実際の文書 [1] では、この差は 3 桁を超える。

この短縮は、組版アルゴリズムの変更や近似によって得られたものではない。使い捨てのプロセスモデルを、状態を保持するランタイムに置き換えたことの帰結である。Web において文書が生きた木となったことが対話的なアプリケーションの基盤となったように、組版においても同じ移行が可能であることを、本稿の実装と測定は示している。

参考文献

- [1] C. Lode. Real-Time LuaTeX: Recompiling Large Documents in 1 ms. *TUGboat*, draft, 2026.
- [2] F. Bour. TeXpresso: Live rendering and error reporting for L^AT_EX. *TUGboat*, 44(2):185–192, 2023.
- [3] D. E. Knuth and M. F. Plass. Breaking Paragraphs into Lines. *Software—Practice and Experience*, 11(11):1119–1184, 1981.
- [4] D. E. Knuth. *The T_EXbook*. Addison-Wesley, 1986.
- [5] S. K. Card, T. P. Moran, and A. Newell. *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates, 1983.
- [6] M. Haug. Fast typesetting with incremental compilation. Master’s thesis, Technische Universität Berlin, 2022.
- [7] Typst. <https://typst.app/> (2026 年 8 月閲覧).
- [8] SwiftLaTeX: The visual L^AT_EX editor. <https://github.com/SwiftLaTeX/SwiftLaTeX>.
- [9] TeXlyre. BusyTeX: T_EX live compiled to WebAssembly. <https://github.com/TeXlyre/texlyre-busytex>, 2026.
- [10] H. T. Thành. *Micro-typographic extensions to the T_EX typesetting system*. PhD thesis, Masaryk University Brno, 2000.
- [11] R. Schlicht. The microtype package: Subliminal refinements towards typographical perfection. <https://ctan.org/pkg/microtype>, 2024.
- [12] W3C. Document Object Model (DOM) Level 1 Specification. W3C Recommendation, 1998.
- [13] LuaTeX development team. *LuaTeX Reference Manual*. <https://www.luatex.org/>.
- [14] TDOM Engine (ソースコード・検証スイート・本稿の計測スクリプト). <https://github.com/Fermion-company/tdom-engine>, 2026.